



Intema System Description for the ASVspoof5 Challenge: Power Weighted Score Fusion

Alexander Kondratev, Ali Aliyev

Intema,
United Arab Emirates

{a.kondratev,a.aliyev}@intema.ai

Abstract

The paper describes Intema submissions to the ASVspoof 5 challenge for Track 1 deepfake detection and Track 2 SASV system (both open). Independent spoofing countersure (CM) and automatic speaker verification (ASV) systems based on Self-supervised learning (SSL) and ResNet-like models were developed. The training process is clearly described for CM and ASV tasks. An aggregation method for this systems is proposed. The final submission for the Track 1 (open) achieved *minDCF* 0.0936, combined system for Track 2 (open) achieved *min a-DCF* 0.1203.

1. Introduction

ASVspoof 5 challenge [1] aims to develop and improve voice biometrics methods such as Automatic Speaker Verification (ASV) and Spoofing Countermeasure (CM). This year's competition was divided into two tracks:

- **Track 1:** which aims to develop a stand-alone CM system.
- **Track 2:** which aims to develop a combined Spoofing-robust Automatic Speaker Verification (SASV) system.

To participate in Track 2, two different approaches can be distinguished:

- Train two separate systems that operate independently and combine them using an additional scoring system.
- Train a single e2e model that will produce values for verification and spoofing.

We decided to follow the first approach and create independent CM and ASV systems. In addition, the CM system from track 1 is reused in track 2.

This paper is structured in the following way. Sections 2 and 3 describe datasets, input features, model architectures, and training details of the Deepfake and Speaker Verification systems respectively. Section 4 gives an overview of our scoring system. Discussions of our results and the conclusions themselves are in Section 5. Conclusions are drawn in Section 6.

2. Deepfake system overview

2.1. Datasets

For training and development for deepfake detection (CM) systems only original ASVspoof5 dataset [2] was used. The training subset contains approximately 78 hours of genuine speech samples and 526 hours of spoof speech samples. The development subset is about half the size of the training subset and contains approximately 61 and 216 hours of genuine and spoof

speech samples respectively. Other speech data was not used in training despite permissions in the open track.

2.2. Input features

For training, variable-length 3-5 seconds audio segments were used with zero-padding. We randomly cropped segments from each utterance in the training dataset. Raw audio was used as inputs for each single CM model. We added only normalization as pre-processing operation after the augmentations had applied. Normalization was calculated by subtracted mean and divided by STD values in each example independently. To test the models, we used first 8-second input segments from each utterance.

2.3. Data augmentation

To augment CM training datasets we used FIR filters [3] and RawBoost [4]. We applied on-the-fly augmentations during the training process. For each training utterance we applied each augmentation with probability 0.5.

- **FIR:** Band-pass finite impulse response filters are applied to mimic speech codecs. Low-pass frequency is randomly sampled from range 3.4-7.5 kHz. High-pass frequency is sampled from 100-400 Hz range. The main idea is to decrease the impact of spectral power in range 0-400 Hz and 3.4-8 kHz which is typical for some codecs. Another idea is that FIR filters are some kind of spectral masking in time domain signal. FIR filtering was implemented by sox sinc filtering called from torchaudio sox-effects.
- **RawBoost:** We use algorithm 4 from the original realization. It applies Linear and non-linear convolutive noise, Impulsive signal-dependent additive noise and Stationary signal-independent additive noise in series. Default hyper-parameters were used.

2.4. DF single models

We utilize pre-trained self-supervised learning (SSL) WavLM-base [5] model as an efficient feature extractor for different audio tasks. WavLM's architecture integrates a CNN encoder with transformer encoders to extract speech features at multiple levels. The CNN encoder is made up of seven temporal convolutional filters. After extraction, these features are further processed by 12 transformer encoders. Each transformer block is designed in accordance with HuBERT's [6] framework, featuring 16 attention heads and a hidden dimension of 768.

Embedding from the last layer is then averaged and passed through the dropout with probability 0.2 and linear layers to get the final prediction. During experiments we used Attentive

Statistic Pooling (ASP) [7] and MLP classifier but they show worse performance on the progress set.

Table 1: DF single model architecture

Module	Output
WavLM-base	$BS \times T \times 768$
Mean reduction	$BS \times 768$
Sequential(Dropout(0.2), Linear)	$BS \times 2$

2.5. Implementation details

We took pre-trained WavLM-base model from HuggingFace [8]. The CNN encoder of WavLM was frozen, given the general applicability of its acoustic features for deepfake detection task. Models were trained using Adamax optimizer with hyperparameters $\beta = [0.9, 0.999]$ and $\text{weight_decay} = 0$. We set cosine learning-rate scheduler to accelerate convergence. For the finetuning process, we set the initial learning rate at 5×10^{-5} .

We have trained a CM models for 10 epoch with an effective batch size of 128. Training batches were formed using weighted random sampler with equal probabilities for bonafide and spoof classes. The training criteria consist of Cross-entropy (CE) loss and One Class (OC)-Softmax [9] with weights [0.9, 0.1] respectively. We used CE loss with equal weights [1.0, 1.0] for both classes. OC loss hyperparameters are $r_{\text{real}} = 0.9$, $r_{\text{fake}} = 0.5$ and $\alpha = 20.0$.

2.6. DF scoring and fusion scheme

For evaluation we used first 8s from utterances. The score for single utterance (cm-score) is a softmax probability for a genuine class.

Our final DF system combines 5 best single models respect to the performance of each single system on the development and progress sets. Fusion was performed on the score level by averaging the single system scores.

3. Speaker verification system overview

As single speaker verification systems for our SASV solution we decided to use 2 ASV architectures: the SJTU X-LANCE Lab System’s ResNet modification [10] and CAM++ [11].

3.1. Datasets

As the basis of our training dataset, we decided to use the Vox-Celeb2 [12] dataset with the addition of a Russian language sub-dataset from Mozilla Common Voice 13.0 [13]. Using a dataset in a different language allows us to avoid a bit of language dependency and prepare for all sorts of scenarios that may arise at different stages of the competition. The ASVSpooF5 dataset is used for development purposes only.

3.2. Input features

As input data for our models, we used 2s random segments of samples from which we extracted mean-normalized 80-dimensional filter-banks with a 25 ms frame length and 10 ms step. For the LMF (Large Margin Fine-tuning) training phase, the chunk size was increased to 6 s.

3.3. Data Augmentation

For augmentation during training, we used the following datasets: Room Impulse Response and Noise Database (RIR)[14] and MUSAN [15]. It should be taken into account that only recordings with music and noise were used from the MUSAN dataset, recordings with human speech were excluded from our dataset for augmentation because of the deterioration of the final model quality when using them. In our training pipeline of verification models, we did not use SpecAugment [16] because it did not help us to improve the models, but only added unnecessary computational complexity during sample preprocessing and degraded the results of the models. Each of the above augmentations were added for each iteration with a probability of 0.6. **Gaussian dithering** is applied to each sample. To artificially increase the number of speakers in our dataset, we also applied speed perturbation [0.9, 1.0, 1.1], which increased the number of speakers by a factor of 3.

3.4. Model architectures

As mentioned above, we used The SJTU X-LANCE Lab System’s ResNet modification as the basis for our model. One of the differences of this model is the removed Max Pooling layer, since we have already processed compressed input data in the form of filter banks and we do not need excessive compression.

But the main difference of this model is in the modified ResNet blocks, unlike the original block, here instead of two convolutions with kernel size = 3x3 layers, we use 2 pointwise convolutions and one convolutional layer with 3x3 kernel size, as shown in Table 3. This approach allows us to extract more information from the data and train our model better, while this modification reduces the number of network parameters and improves the performance of the system.

We also decided to use the new CAM++ architecture, which showed quite similar results to our model and to diversify our fusion of model results, we decided to add it to our pipelines.

Table 2: Original ResNetBlock structure

ResNetBlock-1	Output
$\begin{bmatrix} 3 \times 3, 32 \\ 3 \times 3, 32 \end{bmatrix} \times N$	$80 \times T \times 32$

Table 3: Modified ResNetBlock structure

ResNetBlock-1	Output
$\begin{bmatrix} 1 \times 1, 32 \\ 3 \times 3, 32 \\ 1 \times 1, 128 \end{bmatrix} \times N$	$80 \times T \times 128$

We chose **TSTP** (Temporal Statistical Pooling) as our pooling function, which concatenates the mean and standard deviation of our time-series features. We could have chosen more sophisticated implementations of TSTP, like Attentive Statistics Pooling (ASTP), Multi Head Attentive Statistics Pooling (MHASTP) [17] or Multi Query Multi Head Attentive Statistics Pooling (MQMHASTP) [18], but abandoned this idea to avoid the likelihood of overfitting, as shown in this paper [19] and other problems that could arise while using these methods.

3.5. Training process details

All our models were on batch size=128 with 2 s random chunks using AAM-Softmax [20] loss where scaling factor was set to 32, the first 20 epochs were trained using margin = 0, then from epoch 20 to 40 it exponentially increased to 0.2 and after that kept at 0.2 until the end of the main training phase. Stochastic gradient descent (SGD) with Nesterov Momentum with exponential decrease in learning rate from initial 0.1 to 0.00005, momentum=0.9 and weight decay (L2 penalty) = 0.0001. The main stage of model training lasted 130-160 epochs.

In the second stage we applied Large Margin Fine tuning (LMF) [21], the size of input chunks was increased to 6 s and the batch size was reduced to 64 because of the increased length of input chunks. Margin was increased to a constant 0.5 for 5 epochs. The initial learning rate was 0.0001 and the final learning rate was 0.000025. At the end of each model, TSTP was used as a pooling function.

3.6. ASV scoring and fusion scheme

As scores (asv-score), we counted the cosine distance between each current speaker sample and the current sample, and then averaged the results. Usually, various machine learning methods such as Logistic Regression and others are used to combine scores of different speaker verification models, but due to big differences between our scores on dev and progress, and similar results of our models on the progress part of the challenge, we decided to use a simpler method, averaging the results of the two models. Because of the big differences between the dev and progress parts, we also decided not to use AS-Norm [22] to calibrate the scores of the models. Not using more complex methods of combining and calibrating scores will allow us to avoid overfitting and strong deterioration of results at the next stages.

4. SASV Fusion

Since we prepared independent CM and ASV systems for SASV task, we used two categories of approaches to combine the solutions: cascaded systems and score level fusion.

$$sasv-score = \begin{cases} asv-score & \text{if } cm-score > threshold \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

Cascaded system beginning with an CM binary classifier op followed by the ASV. CM classifier makes a decision based on a optimal *threshold* calculated on a development dataset from track 1 protocol. This is formalized by the equation (1).

$$sasv-score = a * asv-score + (1 - a) * cm-score \quad (2)$$

As basic score level fusion approach we used a score-sum [23] fusion (2) with hyperparameter a which corresponds to the weight in the linear combination between ASV and CM systems.

$$sasv-score = asv-score * cm-score^q \quad (3)$$

The another idea of score level fusion is to multiply scores with raising $cm-score$ to the power of q . Assumes that $q \geq 1$. This operation is allowed in case when $cm-score$ belongs to the interval $[0; 1]$ after softmax operation. Multiplying the CM and ASV makes sense because a only high rates of both scores means target speaker in the terms of SASV tasks. This can be thought of as logical AND operation. Raising to the power of q

allows us to make *cm-score* more categorical, but still continuous on the interval $[0; 1]$, in contrast to making a hard decision on the some fixed threshold. We called this approach the **Power Weighted Score Fusion (PWSF)**.

5. Results and Discussion

5.1. Track 1 open

As shown in Table 4 single fine-tuned WavLM-base models achieve an average of 1.47% EER and 0.0337 minDCF on development. This results are close to the quality metrics calculated on progress set Table 6. Average result for single CM models on progress data is 1.14 EER and 0.0313 minDCF which is close to development metrics. At the same time it was hard to find improvements to the model because the metric on development set did not correlate with the metric on progress set. For example *System₂* has much worse development metrics then other single systems, but is one of the best single systems of progress with minDCF 0.0306.

Table 4: CM single systems performance (dev set)

Model	minDCF	EER
System ₁	0.0292	1.19
System ₂	0.0497	1.99
System ₃	0.0322	1.39
System ₄	0.0285	1.34
System ₅	0.0289	1.43

Most of attack systems in development as shown in Table 5 are detected with $EER \lesssim 0.3\%$. The only difficult systems to recognize are A13 and A15 which ensure a high rate of pooled error.

Table 5: CM EER for attack systems (dev set)

A09	A10	A11	A12	A13	A14	A15	A16	Pooled
0.07	0.06	0.21	0.31	4.23	0.01	1.07	0.02	1.43

Table 6: CM systems performance (progress set)

Model	minDCF	actDCF	C_{llr}	EER
System ₁	0.0329	1.0	0.8028	1.20
System ₂	0.0306	1.0	0.7600	1.08
System ₃	0.0303	1.0	0.7969	1.14
System ₄	0.0323	1.0	0.7985	1.21
System ₅	0.0302	1.0	0.8358	1.07
CM Ensemble	0.0254	1.0	0.8381	0.90

Table 6 demonstrates that the models are not well calibrate and achieved a low rates of actDCF and C_{llr} metrics. We have verified (Table 7) that the use of LLR scores instead of softmax scores allows to achieve much better performance for this metrics. At the same time, since actDCF and C_{llr} are secondary metrics and LLR scoring doesn't not affect the main metric minDCF, we decided to use softmax scores throughout the competition due to the convenience of a limited range of values $[0; 1]$.

The ensemble of single systems has prime metrics 0.0254 minDCF on progress set. This models ensemble chosen to be

Table 7: CM Softmax and LLR scores (progress set)

Model	minDCF	actDCF	C_{llr}	EER
CM Ensemble, Softmax scores	0.0254	1.0	0.8381	0.90
CM Ensemble, LLR scores	0.0252	0.1737	0.2792	0.90

the final submission for evaluation dataset and achieves 0.094 minDCF as shown in Table 8.

Table 8: Track 1 open final results

Model	minDCF	actDCF	C_{llr}	EER
CM Ensemble	0.0936	1.0	0.8874	3.41

The Table 9 shows minDCF for different compression codecs from evaluation subset. The CM system shows performance degradation when using codecs. The minDCF has worsened from 0.0342 to 0.0957 in average for 16 kHz codecs C01-C07 and to 0.1126 for 8 kHz codecs C08-C11. This result can be considered quite acceptable, since the system continues to solve deepfake detection task. In addition, the quality metrics at 8 kHz and 16 kHz are close to each other. We note that degradation appears when the neural codec Encodec [24] (codecs C04 and C07) was applied to the data. We assume that augmentation of training data using neural codecs will significantly reduce performance degradation.

Table 9: Track1 codecs performance

Codec	Bandwith	minDCF
C00	16 kHz	0.0342
C01	16 kHz	0.0610
C02	16 kHz	0.0577
C03	16 kHz	0.0927
C04	16 kHz	0.1440
C05	16 kHz	0.0385
C06	16 kHz	0.0541
C07	16 kHz	0.1877
C08	8 kHz	0.1105
C09	8 kHz	0.1001
C10	8 kHz	0.1867
C11	8 kHz	0.0530
C01-C07	16 kHz	0.0957
C08-C11	8 kHz	0.1126
Pooled	8 & 16 kHz	0.0936

5.2. ASV systems for track 2

During the development of the single ASV systems, we encountered a problem as some samples that were labeled as spoof obviously were generated from speaker audio recordings of target and non-target samples. This led to the fact that even though the spoof labeled samples should have had negative final results, but the verification system produced positive results, which led to a lot of errors because there was a lot of generated data in the dataset. For this reason all metrics for Table 10 for development part of dataset were made without samples labeled as ‘‘spoof’’, since there are no labels for speaker verification. Therefore the number of rows in trials was reduced from 282 456 to 172 840 rows. All results in the Table 10 for minDCF were calculated with the following values: $p_{\text{target}}=0.9405$, $C_{\text{miss}}=1$, $C_{\text{fa}}=10$.

Table 10: Track 2 ASV systems performance (dev set)

Model	minDCF	EER
(1) ResNet-34	0.0315	1.410
(2) ResNet-152	0.0286	1.282
(3) CAM++	0.0328	1.399
(1) + (2)	0.0293	1.281
(1) + (3)	0.0303	1.322
(2) + (3)	0.0284	1.275
(1) + (2) + (3)	0.0288	1.281

Table 11: Single models performance on VoxCeleb-1 dataset

Dataset	Metric	ResNet-34	ResNet-152	CAM++
Vox1-O	minDCF	0.0260	0.0188	0.0220
	EER	1.029	0.753	0.870
Vox1-E	minDCF	0.0283	0.0221	0.0247
	EER	1.099	0.860	0.959
Vox1-H	minDCF	0.0474	0.0372	0.0455
	EER	1.840	1.449	1.770

As you can see from Table 10 and Table 11, the models behave quite expectedly on the dev part of the challenge dataset and on the VoxCeleb-1 dataset., the largest model ResNet-152 shows the best results for the single model with minDCF 0.0286 and EER 1.282, and the fusion of CAM++ and ResNet-152, is the most successful fusion models with minDCF 0.0284 and EER 1.275. However, as can be seen in Table 12, when we tried to use ResNet-152 in the progress part, we encountered a degradation of the min a-DCF and EER metrics. At the same time, small models such as ResNet-34 and CAM++ behave much more stable in the progress part relative to the dev part of the dataset. Therefore, it was decided to use the fusion of ResNet-34 and CAM++ with with min a-DCF 0.0732 and EER 3.66. on progress set in the future.

Table 12: Track 2 ASV systems performance combined with CM (progress set)

Model	min a-DCF	t-EER
(1) ResNet-34 + CM	0.0749	3.84
(2) ResNet-152 + CM	0.1102	4.24
(3) CAM++ + CM	0.0768	3.86
(1) + (2) + CM	-	-
(1) + (3) + CM	0.0732	3.66
(2) + (3) + CM	-	-
(1) + (2) + (3) + CM	-	-

In Table 12, we used the same CM system for all submissions, which was the ensemble of all single systems from Table 4.

Since in min a-DCF $C_{\text{miss}}=1$, $C_{\text{fa}}=10$, which means that, cost of a false alarm is greater than the cost of a missed detection, guiding the system to more strictly minimize false alarms than missed detections. So increasing margin in AAM-Softmax Loss and also weight decay in SGD maybe can lead to reducing the number of false alarms.

5.3. Track 2 open

As the ASV final system we use ensemble (1) + (2). The CM final system is the same as in the track 1. Thus, the final SASV system includes 7 single systems. We explored different approaches from section 4 to combine scores. Optimal hyperparameter values found on the development set. The Table 13 show the performances for different methods on the development and progress sets. The results show that power weighted score fusion is significantly better than cascaded system. This fusion method has proven its effectiveness because cm-scores from SSL models have a very categorical distribution. Most deepfake utterances have a scores slightly greater than zero and and, on the contrary, examples of bonafide have scores close to the value 1.

The difference between PWSF method and weighted sum is not that significant on the progress set. For the final submission we have choosen power weighted score fusion method because it is not as sensitive to the choice of hyperparameter as weighted sum fusion.

Table 13: SASV fusion methods

Fusion type	Hyperparameter	min a-DCF	
		dev	progress
Cascade (1)	threshold=0.99	0.0407	0.0775
Weighted sum (2)	a=0.01	0.0354	0.0697
PWSF (3)	q=1000	0.0297	0.0692

The ensemble of independent ASV and CM systems combined with proposed PWSF achieves 0.1203 min a-DCF as shown in the Table 14. It can be concluded that the presented method is efficient, although it imposes restriction on the input CM scores to be probability like in range [0; 1].

Table 14: Track 2 open final results

Model	min a-DCF	min t-DCF	t-EER
ASV & CM Ensemble	0.1203	0.4291	4.54

Table 15: Track2 codecs performance

Codec	Bandwith	min a-DCF
C00	16 kHz	0.0759
C01	16 kHz	0.1011
C02	16 kHz	0.0889
C03	16 kHz	0.1202
C08	8 kHz	0.1375
C09	8 kHz	0.1204
C10	8 kHz	0.2032
C11	8 kHz	0.0954
C01-C03	16 kHz	0.1034
C08-C11	8 kHz	0.1391
Pooled	8 & 16 kHz	0.1203

As you can see in Table 15, the min a-DCF metric increased compare no-codec condition for all codecs in the dataset by between 0.0130 and 0.1280 for different codecs. The relative degradation of the min a-DCF metric is not as significant as in the 1 track, but the quality is noticeably worse for 8 kHz then

for 16 kHz codecs. Our verification models were trained only on original full-band audio files, and during the training we did not use dynamic changes of the sampling rate of files and application of compression used in different codecs.

6. Conclusion

In our paper we described, architectures and training methods of two separate systems: ensemble of 5 WavLM-base models for Deepfake Detection (CM) and ensemble of ResNet34 and CAM++ models for Automatic Speaker Verification (ASV). The results between the two systems were combined using the proposed Power Weighted Score Fusion method. Our final systems achieved **minDCF 0.0936** on **Track 1** and **min a-DCF 0.1203** on **Track 2**.

7. References

- [1] Héctor Delgado, Nicholas Evans, Jee-weon Jung, Tomi Kinnunen, Ivan Kukanov, Kong Aik Lee, Xuechen Liu, Hye-jin Shim, Md Sahidullah, Hemlata Tak, et al., “Asvspoof 5 evaluation plan,” 2024.
- [2] Xin Wang, Héctor Delgado, Hemlata Tak, Jee-weon Jung, Hye-jin Shim, Massimiliano Todisco, Ivan Kukanov, Xuechen Liu, Md Sahidullah, Tomi Kinnunen, Nicholas Evans, Kong Aik Lee, and Junichi Yamagishi, “ASVspoof 5: Crowdsourced speech data, deepfakes, and adversarial attacks at scale,” in *ASVspoof Workshop 2024 (accepted)*, 2024.
- [3] Anton Tomilov, Aleksei Svishchev, Marina Volkova, Artem Chirkovskiy, Alexander Kondratev, and Galina Lavrentyeva, “Stc antispoofing systems for the asvspoof2021 challenge,” in *Proc. ASVspoof 2021 Workshop*, 2021, pp. 61–67.
- [4] Hemlata Tak, Madhu Kamble, Jose Patino, Massimiliano Todisco, and Nicholas Evans, “Rawboost: A raw data boosting and augmentation method applied to automatic speaker verification anti-spoofing,” in *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2022, pp. 6382–6386.
- [5] Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, Jinyu Li, Naoyuki Kanda, Takuya Yoshioka, Xiong Xiao, et al., “Wavlm: Large-scale self-supervised pre-training for full stack speech processing,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.
- [6] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed, “Hubert: Self-supervised speech representation learning by masked prediction of hidden units,” 2021.
- [7] Koji Okabe, Takafumi Koshinaka, and Koichi Shinoda, “Attentive statistics pooling for deep speaker embedding,” in *Interspeech 2018*. Sept. 2018, ISCA.
- [8] Microsoft, “Wavlm: A unified framework for self-supervised learning of speech representation,” huggingface.co/microsoft/wavlm-base, 2024, Accessed: 2024-07-01.
- [9] You Zhang, Fei Jiang, and Zhiyao Duan, “One-class learning towards synthetic voice spoofing detection,” *IEEE Signal Processing Letters*, vol. 28, pp. 937–941, 2021.

- [10] Zhengyang Chen, Bei Liu, Bing Han, Leying Zhang, and Yanmin Qian, “The sjtu x-lance lab system for cnsrc 2022,” 2023.
- [11] Hui Wang, Siqi Zheng, Yafeng Chen, Luyao Cheng, and Qian Chen, “Cam++: A fast and efficient network for speaker verification using context-aware masking,” in *Proceedings of the Annual Conference of the International Speech Communication Association, INTERSPEECH*, 2023, vol. 2023-August.
- [12] Arsha Nagrani, Joon Son Chung, and Andrew Senior, “VoxCeleb: A large-scale speaker identification dataset,” in *Interspeech 2017*, aug 2017, ISCA.
- [13] Rosana Ardila, Megan Branson, Kelly Davis, Michael Henretty, Michael Kohler, Josh Meyer, Reuben Morais, Lindsay Saunders, Francis M. Tyers, and Gregor Weber, “Common voice: A massively-multilingual speech corpus,” *CoRR*, vol. abs/1912.06670, 2019.
- [14] “Room impulse response and noise database,” 2017, Accessed: 2024-07-01.
- [15] David Snyder, Guoguo Chen, and Daniel Povey, “Musan: A music, speech, and noise corpus,” *arXiv preprint arXiv:1510.08484*, 2015.
- [16] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le, “SpecAugment: A simple data augmentation method for automatic speech recognition,” in *Interspeech 2019*, Sept. 2019, ISCA.
- [17] Miquel India, Pooyan Safari, and Javier Hernando, “Self multi-head attention for speaker recognition,” 2019.
- [18] Miao Zhao, Yufeng Ma, Yiwei Ding, Yu Zheng, Min Liu, and Minqiang Xu, “Multi-query multi-head attention pooling and inter-topk penalty for speaker verification,” 2021.
- [19] Ali Aliyev, “Intema system description for the voxceleb speaker recognition challenge 2023,” in *VoxCeleb Speaker Recognition Challenge Workshop 2023*, 2023.
- [20] Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou, “Arcface: Additive angular margin loss for deep face recognition,” in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4685–4694.
- [21] Jenthe Thienpondt, Brecht Desplanques, and Kris Demuynck, “The IDLAB voxsrc-20 submission: Large margin fine-tuning and quality-aware score calibration in DNN based speaker verification,” *CoRR*, vol. abs/2010.11255, 2020.
- [22] Pavel Matějka, Ondřej Novotný, Oldřich Plchot, Lukáš Burget, Mireia Diez Sánchez, and Jan Černocký, “Analysis of Score Normalization in Multilingual Speaker Recognition,” in *Proc. Interspeech 2017*, 2017, pp. 1567–1571.
- [23] Hye-jin Shim, Hemlata Tak, Xuechen Liu, Hee-Soo Heo, Jee-weon Jung, Joon Son Chung, Soo-Whan Chung, Ha-Jin Yu, Bong-Jin Lee, Massimiliano Todisco, et al., “Baseline systems for the first spoofing-aware speaker verification challenge: score and embedding fusion,” *arXiv preprint arXiv:2204.09976*, 2022.
- [24] Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi, “High fidelity neural audio compression,” *arXiv preprint arXiv:2210.13438*, 2022.